Software Project Management (Lect 9)

Prof. H S Behera

Organization of this Lecture:

- Overview of Software Project Management (SPM)
- Introduction to Project Planning
- Software Cost Estimation
  - ✓Cost Estimation Models
  - ✓Software Size Metrics
  - ✓Empirical Estimation
  - ✓Heuristic Estimation
  - ✓COCOMO
- Staffing Level Estimation
- Effect of Schedule Compression on Cost
- Summary

Overview to Software Project Management (SPM)

- SPM is the planning, organizing, monitoring, and control of software development processes to deliver high-quality, on-budget, and on-schedule products i.e. within specific time and budget constraints.
- Key activities include defining project scope, managing risks, allocating resources, and ensuring stakeholder satisfaction through methodologies.

Software Project Management

- **Software Project Management** (SPM), **is the discipline of planning, monitoring, and controlling software projects to achieve specific targets while optimizing resources like time, money, and personnel**

Introduction to Project planning (PP)

- Project planning is the critical phase of project management, occurring after initiation and before execution, where the roadmap for a project is created. It involves defining goals, setting, and outlining tasks, resources, and schedules to ensure project success.
- This phase ensures stakeholders are aligned, risks are mitigated, and resources are efficiently allocated by reducing the scope creep.

Introduction to Project planning

- **Proper project planning minimizes risks of failure, prevents budget overruns, and ensures that team members understand their responsibilities, reducing uncertainty, project stress, and costly rework**.
- It creates a structured, actionable plan, which serves as a guide throughout the project lifecycle and increases overall team efficiency and project success.

Introduction to Project planning (PP)

- **Project planning is essential for ensuring projects are completed on time, within budget, and to the required standard**.
- Key benefits include improved resource allocation, early risk identification, clear team accountability, and enhanced communication among stakeholders.
- A solid plan reduces uncertainty, prevents scope creep and increases overall team efficiency and project success.

Introduction to PP

- Many software projects fail:
  - due to faulty project management practices:
    - ✓It is important to learn different aspects of software project management.

Introduction to PP

- Goal of software project management:
  - ✓enable a group of engineers to work efficiently towards successful completion of a software project.

Responsibility of Project Managers

- Project proposal writing,
- Project cost estimation,
- Scheduling,
- Project staffing,
- Project monitoring and control,
- Software configuration management,
- Risk management,
- Managerial report writing and presentations, etc.

Introduction to PP

- A project manager's activities are varied:
  - **can be broadly classified into:**
    - ✓**project planning,**
    - ✓**project monitoring and control activities.**

Project Planning

- Once a project is found to be feasible,
  - ✓project managers undertake Project Planning.

Project Planning Activities

- Estimation:
  - Effort, cost, resource, and project duration
- Project scheduling:
- Staff organization:
  - staffing plans
- Risk handling:
  - identification, analysis, and abatement procedures
- Miscellaneous plans:
  - quality assurance plan, configuration management plan, etc.

Project Management Plan

Project planning

- Requires utmost care and attention --- Commitments to unrealistic time and resource estimates result in:
  - ✓Irritating delays.
  - ✓Customer dissatisfaction
  - ✓Adverse affect on team morale
    - poor quality work
  - ✓Project failure.

Sliding Window Planning

- **Sliding Window Planning**, is a project management approach where planning is done iteratively over several stages rather than in one single, detailed, early step.
- This technique helps protect managers from making large, risky, and premature commitments before they have enough information.

Sliding window planning

- **Sliding window planning is a project management approach where planning and development occur in overlapping or sequential phases, allowing for continuous adaptation and refinement.**
- It breaks long projects into manageable, frequently updated, and iterative cycles, ensuring the plan remains relevant as requirements evolve, rather than adhering to a rigid, upfront structure.

Sliding window planning

- **Sliding window planning** is an iterative project management technique where planning occurs in overlapping stages or " windows," allowing teams to refine details as more information becomes available. It is ideal for long-term projects with high uncertainty, as it enables adjusting to evolving requirements, risks, and progress.

Sliding Window Planning

- Involves project planning over several stages:
  - ✓Protects managers from making big commitments too early.
  - ✓More information becomes available as project progresses.
    - Facilitates accurate planning

SPMP Document

- **A Software Project Management Plan (SPMP)** is a formal document detailing the planning, scheduling, resource allocation, and control mechanisms for a software project.
- **Based on IEEE standards, it defines the project's purpose/objectives, scope, schedules, deliverables, budget, & risks management to guide teams members from conception to final delivery. It acts or serving as a roadmap for project managers, ensuring development and alignment on goals.**

SPMP Document

- After planning is complete:
  - **Document the plans:**
    - ✓ **in a Software Project Management Plan(SPMP) document.**
    - ✓ A **SPMP** is a living document that serves as the central "blueprint" for a software project. It outlines the technical and managerial processes required to deliver a high-quality product on time & within budget.

Organization of SPMP Document

- **Introduction** (Objectives, Major Functions, Performance Issues, Management and Technical Constraints)
- **Project Estimates** (Historical Data, Estimation Techniques, Effort, Cost, and Project Duration Estimates)
- **Project Resources Plan** (People, Hardware and Software, Special Resources)
- **Schedules** (Work Breakdown Structure, Task Network, Gantt Chart Representation, PERT Chart Representation)
- **Risk Management Plan** (Risk Analysis, Risk Identification, Risk Estimation, Abatement Procedures)
- **Project Tracking and Control Plan**
- **Miscellaneous Plans** (Process Tailoring,Quality Assurance)

Software Cost Estimation

- **Software cost estimation is a crucial project planning activity that determines the effort, time, and cost required to develop software.**
- It primarily uses empirical and heuristic techniques like the **COCOMO** model, focusing on human effort (**person-months**) rather than just hardware, aimed at reducing the high failure rate of software projects.

Software Cost Estimation

- **Software cost estimation** is the systematic process of predicting the effort (person-hours), duration, and financial resources needed to develop and maintain software.
- It typically involves determining software size, complexity, and team expertise to create realistic budgets and schedules, using techniques like algorithmic modeling (COCOMO), expert judgment, and analogy-based methods.

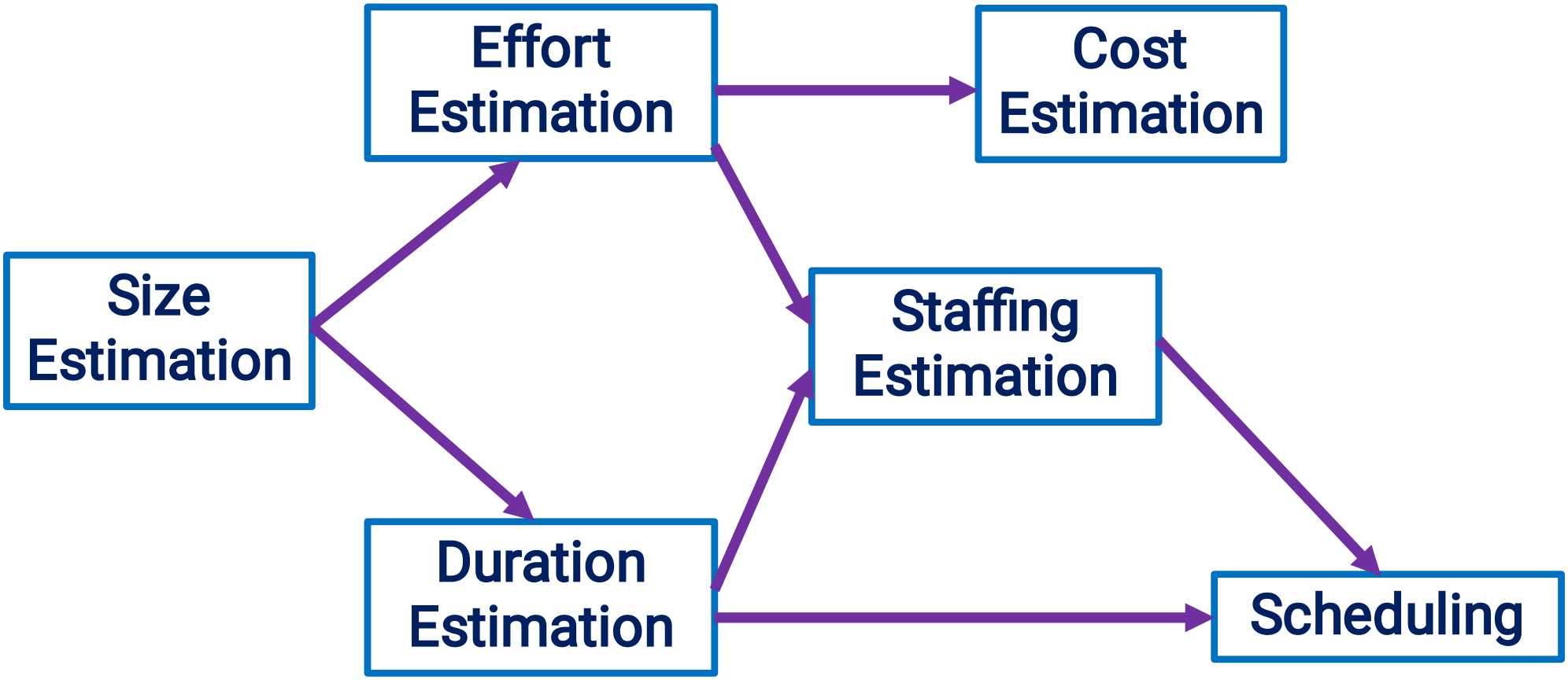
Software Cost Estimation

- **Software cost estimation techniques** predict the effort, time, and financial resources required for development, categorized into **empirical (expert judgment, Delphi), heuristic (COCOMO), and analytical models**.
- Key approaches include top-down (system-level) & bottom-up (component-level) estimation, alongside modern techniques like function point analysis & machine learning.

Software Cost Estimation

- Determine size of the product.
- From the size estimate,
  - determine the effort needed.
- From the effort estimate,
  - determine project duration, and cost.

Software Cost Estimation



Software Cost Estimation

- Three main approaches to estimation:
  - Empirical
  - Heuristic
  - Analytical

Software Cost Estimation Techniques

- **Empirical techniques:**
  - an educated guess based on past experience.
- **Heuristic techniques:**
  - assume that the characteristics to be estimated can be expressed in terms of some mathematical expression.
- **Analytical techniques:**
  - derive the required results starting from certain simple assumptions.

Empirical Techniques

- **Empirical techniques for software cost estimation involve using historical data from previous projects to estimate effort and cost, primarily utilizing Expert judgment and Delphi techniques.**
- These approaches focus on **estimating product size (LOC or Function Points) first**, then converting it to effort, duration, & staffing requirements based on past experiences.

Heuristic Techniques

- **Heuristic techniques** in software cost estimation, **are mathematical models** that use historical data to establish relationships between software parameters (e.g., size) and project costs (effort/time). They offer a structured, quick way to estimate by automating calculations after identifying input variables. **The Constructive Cost Model (COCOMO) is the primary heuristic model.**

Key aspects of Heuristic Techniques include:

- **Basis:** They rely on mathematical expressions and empirical data from previous projects.
- COCOMO (Constructive Cost Model): A prominent model used to estimate effort, duration, and cost based on software size (typically in KLOC - Thousand Lines of Code).
  - ✓ **Basic COCOMO**: Estimates cost based solely on product size.
  - ✓ **Intermediate COCOMO**: Refines estimates using 15 cost drivers (product, hardware, personnel, project attributes).

Key aspects of Heuristic Techniques Contd..

- **Types:**
  - **Single variable models:** Uses one parameter, such as size, to predict costs.
  - **Multi-variable models:** Uses multiple parameters to increase accuracy.
- **Process:**
  - Estimate the software size (e.g., LOC or Function Points).
  - Apply formula-based models to calculate effort (person-months) and schedule.
  - Adjust results based on cost drivers (complexity, experience).
- These techniques are popular because they provide objective, repeatable estimations, unlike expert judgment, which is subjective.

Analytical techniques

- **Analytical techniques** in software cost estimation, are mathematical models (e.g., COCOMO, regression analysis) that derive effort, time, and cost from project parameters like size and complexity.
- These techniques rely on historical data and quantitative formulas, enabling objective, reproducible estimations rather than relying solely on subjective expert judgment.

Key Analytical Techniques & Concepts

- COCOMO (Constructive Cost Model): A widely used mathematical model that calculates effort (E) and development time (D) based on the size of the product in KLOC (Kilo Lines of Code) and the project category (Organic, Semidetached, Embedded).
  - Basic COCOMO Formula: Effort = $a1 \times (KLOC)^{a2} PM$
  - Dev. Time Formula: $T_{dev} = b_1 \times (Effort)^{b2}$ Month
  - a1, a2, b1, b2 are constant for each category of software products
- Regression Models: These analyze the relationship between historical project data (cost drivers) to create models that predict the effort required for new projects.

Key Analytical Techniques & Concepts Contd..

- **Process-Based Estimation:** Breaks down the project into specific processes, activities, and tasks, applying cost rates to each component to arrive at a total estimate.
- **Size Metric Modeling (LOC/Function Points):**

  Analytical methods typically require an input size, usually measured in Lines of Code (LOC) or Function Points (FP), which is then fed into the model.
- **Multi-variable Models:** Use multiple parameters (e.g., team experience, complexity, performance constraints) to predict cost, which are more precise than single-variable formulas.
- These analytical approaches aim to reduce the inaccuracy found in subjective, empirical methods by providing a structured, quantifiable methodology for estimation.

Software Size Metrics

- software size metrics quantify project scope for effort and cost estimation, with Lines of Code (LOC) and Function Points (FP) being the primary methods. LOC counts source instructions but is language-dependent, while FP measures functionality from user requirements (inputs/outputs).

Software Size Metrics

- Lines of Code (LOC): A direct, simple measure of the total instructions in the software, though it is hard to estimate early and varies by programming language.
- Function Point (FP) Analysis: An indirect measure that estimates size based on functionality, including user inputs, outputs, inquiries, internal files, and external interfaces, weighted by complexity.
- **LOC is often used for historical data and finished projects, while FP is preferred for early-stage estimation because it can be calculated from specifications before code is written.**

Software Size Metrics

- **LOC (Lines of Code):**
  - ✓Simplest and most widely used metric.
  - ✓Comments and blank lines should not be counted.

Disadvantages of Using LOC

- Size can vary with coding style.
- Focuses on coding activity alone.
- Correlates poorly with quality and efficiency of code.
- Penalizes higher level programming languages, code reuse, etc.

Disadvantages of Using LOC (cont...)

- Measures lexical/textual complexity only.
  - does not address the issues of structural or logical complexity.
- Difficult to estimate LOC from problem description.
  - So not useful for project planning

Function Point Metric

- Overcomes some of the shortcomings of the LOC metric
- Proposed by Albrecht in early 80's:
  - FP=4 * Number of inputs + 5 * No. Of Outputs + 4 * No. Of inquiries + 10 * F No of Files + 10 * No of interfaces
- Input:
  - A set of related inputs is counted as one input.

Function Point Metric

- Output:
  - A set of related outputs is counted as one output.
- Inquiries:
  - Each user query type is counted.
- Files:
  - Files are logically related data and thus can be data structures or physical files.
- Interface:
  - Data transfer to other systems.

Function Point Metric (CONT.)

- Suffers from a major drawback:
  - the size of a function is considered to be independent of its complexity.
- Extend function point metric:
  - Feature Point metric:
  - considers an extra parameter:
    - Algorithm Complexity.

Function Point Metric (CONT.)

- **Proponents claim:**
  - FP is language independent.
  - Size can be easily derived from problem description
- **Opponents claim:**
  - it is subjective --- Different people can come up with different estimates for the same problem.

Key Empirical Techniques

- **Expert Judgment:** Relies on experienced personnel who analyze the new project's requirements against past projects to make an educated guess on effort, cost, and schedule.
- **Delphi Cost Estimation:** A structured, anonymous group approach where experts refine their estimates over several rounds, reducing the influence of extreme opinions to reach a consensus.

Empirical Size Estimation Techniques

- **Expert Judgement:**
  - An euphemism for guess made by an expert.
  - Suffers from individual bias.
- **Delphi Estimation:**
  - overcomes some of the problems of expert judgement.

Expert judgement

- **Experts divide a software product into component units:**
  - e.g. GUI, database module, data communication module, billing module, etc.
- **Add up the guesses for each of the components.**

Delphi Estimation:

- **Team of Experts and a coordinator.**
- **Experts carry out estimation independently:**
  - mention the rationale behind their estimation.
  - coordinator notes down any extraordinary rationale:
    - circulates among experts.

Delphi Estimation:

- Experts re-estimate.
- Experts never meet each other
  - to discuss their viewpoints.

Heuristic Estimation Techniques

- **Single Variable Model:**
  - Parameter to be Estimated=C1(Estimated Characteristic) * e x Pow (d1)where e is characteristics of software (independent variable) i.e EP= $c1*e^{d1}$
- **Multivariable Model:**
  - Assumes that the parameter to be estimated depends on more than one characteristic.
  - Parameter to be Estimated=C1(Estimated Characteristic)*$e1^{d1}$+ C2(Estimated Characteristic)* e^{d2} +… (where e1,e2 are basic independent characteristics of software already estimated and c1,c2, d1,d2 are constants).
  - Usually more accurate than single variable models.

COCOMO Model

- COCOMO (COnstructive Cost MOdel) proposed by Boehm.
- Divides software product developments into 3 categories:
  - Organic
  - Semidetached
  - Embedded

COCOMO Product Classes

- Roughly correspond to:
  - application, utility and system programs respectively.
    - ✓Data processing and scientific programs are considered to be application programs.
    - ✓Compilers, linkers, editors, etc., are utility programs.
    - ✓Operating systems and real-time system programs, etc. are system programs.

Elaboration of Product Classes

- **Organic:**
  - Relatively small groups
    - working to develop well-understood applications.
- **Semidetached:**
  - Project team consists of a mixture of experienced and inexperienced staff.
- **Embedded:**
  - The software is strongly coupled to complex hardware, or real-time systems.

COCOMO Model (CONT.)

- For each of the three product categories:
  - From size estimation (in KLOC), Boehm provides equations to predict:
    - ✓project duration in months
    - ✓effort in programmer-months
- Boehm obtained these equations:
  - examined historical data collected from a large number of actual projects.

COCOMO Model (CONT.)

- Software cost estimation is done through three stages:
  - Basic COCOMO,
  - Intermediate COCOMO,
  - Complete COCOMO.

Basic COCOMO Model (CONT.)

- **Gives only an approximate estimation:**
  - **Effort = a1 x (KLOC) Pow (a2)**
  - **Tdev = b1 x (Effort) pow (b2)**
    - ✓KLOC is the estimated kilo lines of source code,
    - ✓a1,a2,b1,b2 are constants for different categories of software products,
    - ✓Tdev is the estimated time to develop the software in months,
    - ✓Effort estimation is obtained in terms of person months (PMs).

Development Effort Estimation

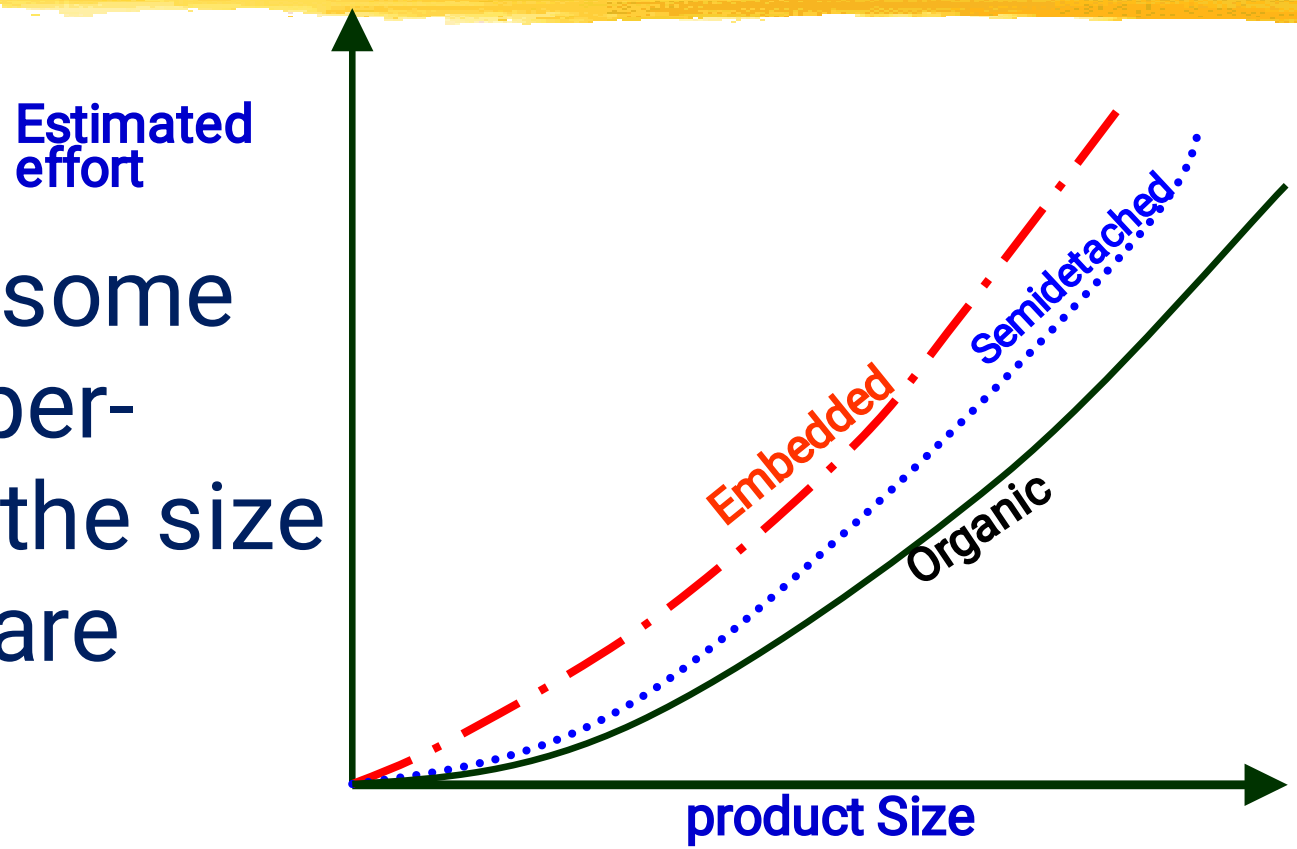
- Organic :
  - Effort = 2.4 (KLOC) pow (1.05) PM
- Semi-detached:
  - Effort = 3.0(KLOC) pow (1.12) PM
- Embedded:
  - Effort = 3.6 (KLOC) pow (1.20)PM

Development Time Estimation

- **Organic:**
  - Tdev = 2.5 (Effort) pow (0.38) Months
- **Semi-detached:**
  - Tdev = 2.5 (Effort) pow (0.35) Months
- **Embedded:**
  - Tdev = 2.5 (Effort) pow (0.32) Months

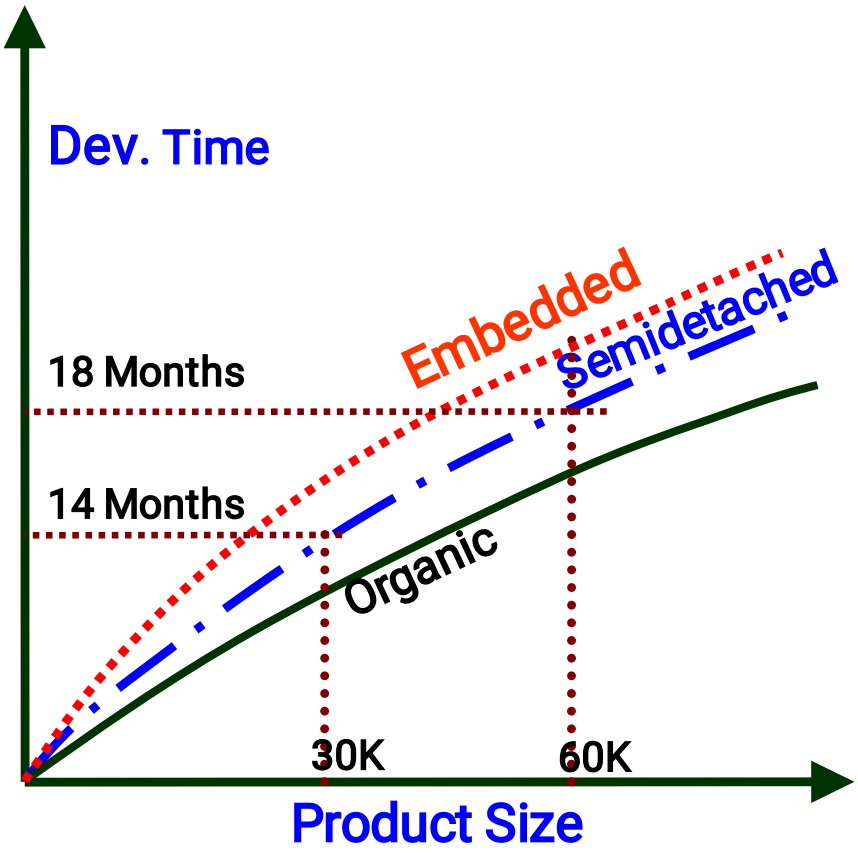
Basic COCOMO Model (CONT.)

- Effort is some what super-linear in the size of software product.



Basic COCOMO Model (CONT.)

- Development time
  - sublinear function of product size.
- When product size increases two times,
  - development time does not double but rises moderately.
- Time taken:
  - almost same for all the three product categories.



Basic COCOMO Model (CONT.)

- Development time does not increase linearly with product size:
  - For larger products more parallel activities can be identified:
    - can be carried out simultaneously by a number of engineers.

Basic COCOMO Model (CONT.)

- Development time is roughly the same for all the three categories of products:
  - **For example, a 60 KLOC program can be developed in approximately 18 months**
    - regardless of whether it is of organic, semi-detached, or embedded type.
  - **There is more scope for parallel activities for system and application programs,**
    - than utility programs.

Example

- The size of an organic software product has been estimated to be 32, 000 lines of source code. Assume that the avg salary of SE is 15,000 per month. Determine the effort required to develop software product and the normal dev. time

.....

- Effort = 2.4*(32) pow (1.05) = 91 PM
- Nominal development time = 2.5*(91) pow (0.38) = 14 months
- Cost required to develop the product = 14 x 15000 = Rs. 210,000

Intermediate COCOMO

- Basic COCOMO model assumes
  - effort and development time depend on product size alone.
- However, several parameters affect effort and development time:
  - ✓Reliability requirements
  - ✓Availability of CASE tools and modern facilities to the developers
  - ✓Size of data to be handled

Intermediate COCOMO

- **For accurate estimation,**
  - the effect of all relevant parameters must be considered:
  - Intermediate COCOMO model recognizes this fact:
    - ✓refines the initial estimate obtained by the basic COCOMO by using a set of 15 cost drivers (multipliers).

Intermediate COCOMO (CONT.)

- If modern programming practices are used,
  - initial estimates are scaled downwards.
- If there are stringent reliability requirements on the product :
  - initial estimate is scaled upwards.

Intermediate COCOMO (CONT.)

- Rate different parameters on a scale of one to three:
  - Depending on these ratings,
    - ✓multiply cost driver values with the estimate obtained using the basic COCOMO.

Intermediate COCOMO (CONT.)

- **Cost driver can be classified of being attributes of the following items:**
  - ✓ **Product:** Inherent complexity of the product, reliability requirements of the product, etc.
  - ✓ **Computer:** Execution time, storage requirements, etc.
  - ✓ **Personnel:** Experience of personnel, etc.
  - ✓ **Development Environment:** Sophistication of the tools used for software development.

Shortcoming of basic and intermediate COCOMO models

- Both models:
  - consider a software product as a single homogeneous entity:
  - However, most large systems are made up of several smaller sub-systems.
    - Some sub-systems may be considered as organic type, some may be considered embedded, etc.
    - for some the reliability requirements may be high, and so on.

Complete COCOMO

- Cost of each sub-system is estimated separately.
- Costs of the sub-systems are added to obtain total cost.
- Reduces the margin of error in the final estimate.

Complete COCOMO Example

- **A distributed Management Information System (MIS) for an organization having offices at several places across the country can have the following sub components:**
  - **Database part (semi-detached)**
  - **Graphical User Interface (GUI) part (organic)**
  - **Communication part (embedded)**
- Costs of the components are estimated separately:
  - **summed up to give the overall cost of the system.**

SUMMARY

Summary of Technique Comparison

Technique	Best Used For...	Main Strength	Main Weakness
COCOMO	Large, well-defined projects	Repeatable and objective	Needs accurate size input (KLOC)
Function Points	Business applications	Independent of programming language	Requires training to count points
Expert Judgement	Small or unique projects	Captures "gut feeling" and nuance	Can be biased or inconsistent
Analogy	Projects similar to past ones	Grounded in actual history	Requires a good database of past projects
Delphi	Minimising group bias	Reaches consensus through logic	Time-consuming

Halstead's Software Science

- Halstead’s Software Science, provides quantitative measures for software complexity, size, and development effort based on operator and operand counts.
- It utilizes metrics like program vocabulary, length, volume, difficulty, and effort to estimate development time and errors, allowing for analysis of programs using basic code counts.
- Halstead's model is valuable for its ability to calculate metrics directly from source code or design components.

Halstead's Software Science

- An analytical technique to estimate:
  - size,
  - development effort,
  - development cost of software product.

Halstead's Software Science

- **Halstead used a few primitive program parameters:**
  - ✓number of operators and operands
- **Derived expressions for:**
  - ✓Over-all program length,
  - ✓Program volume
  - ✓potential minimum volume
  - ✓actual volume,
  - ✓language level,
  - ✓effort, and
  - ✓development time.

Staffing Level Estimation

- **Staffing Level Estimation** determines the number of personnel needed over a project's life cycle, typically using the Rayleigh curve to model effort distribution. Staffing peaks during implementation and testing, with smaller teams required during planning and maintenance, often calculated after estimating total effort (PM) and development time (t_d).

Staffing Level Estimation

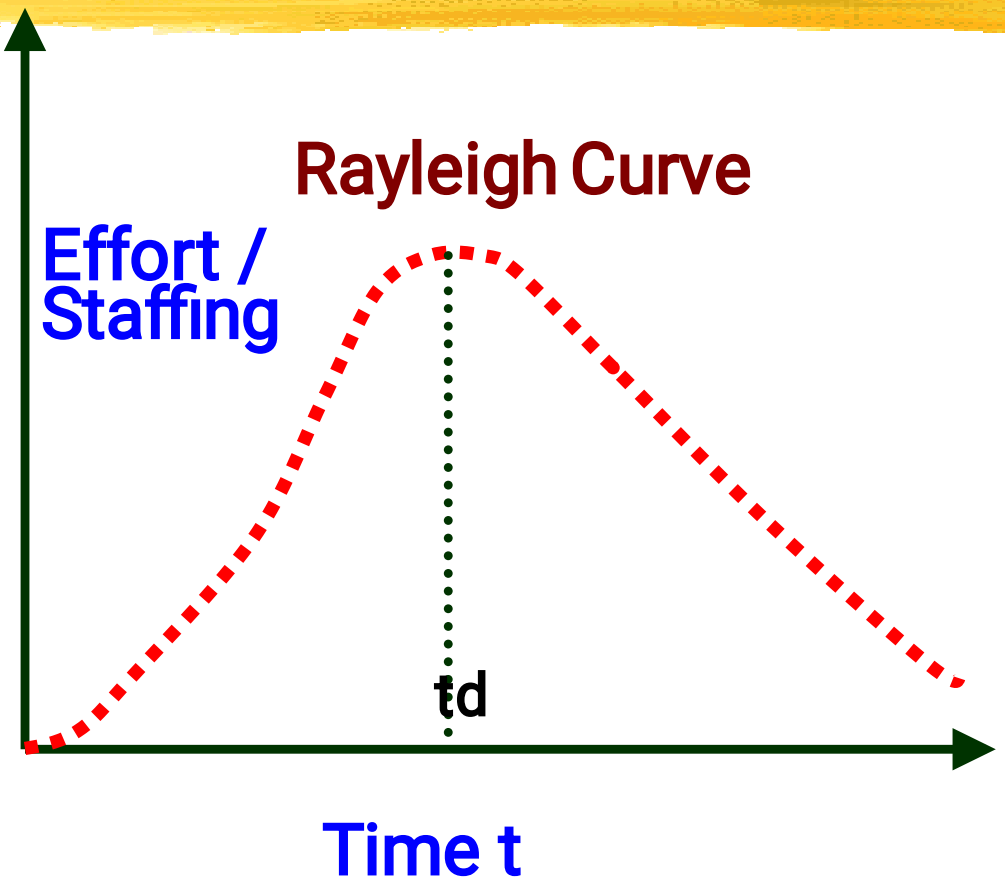
- Number of personnel required during any development project:
  - not constant.
- Norden in 1958 analyzed many R&D projects, and observed:
  - Rayleigh curve represents the number of full-time personnel required at any time.

Rayleigh Curve

- **Staffing needs are not constant**; they follow a specific distribution over time. This is modeled using the **Rayleigh-Norden-Putnam Curve**.
- **Non-Linearity**: Staffing level is not a simple division of effort by time (**e.g., 100 person-months/10 months ≠10 people consistently**).
- **Staffing Distribution:**
  - **Early Phase:** Requires a small, highly skilled team for planning and requirements analysis.
  - **Middle Phase:** Personnel levels peak during the **implementation (coding)** and **testing** stages.
  - **Final Phase:** Staffing tapers off as the project moves toward deployment and maintenance.

Rayleigh Curve

- Rayleigh curve is specified by two parameters:
  - td the time at which the curve reaches its maximum
  - K, the total area under the curve.
- L=f(K, td)



Mathematical Model

- **The staffing level at any time can be expressed by the equation:**

$$E = k/t_d^2 .t.e^{-t2} / 2t_d^2$$

- Where: E = Effort (number of engineers/staff) required at time t.
- K = Total effort expended (area under the curve) .
- t_d = Time at which the staffing level reaches its maximum value.

Rayleigh Curve Contd.

- **Norden represented the Rayleigh Curve by the following equation:**

  E = K/ td pow (2) * t * e pow (-t pow(2)/ 2td pow(2)

- Where E is the effort required at time t. E is an indication of the number of engineers (or the staffing level) at any particular time during the duration of project. K is the area under the curve, td is the time at which the curve attains its maximum value.
- The result of Noden are applicable to general R&D project.

Putnam’s Work:

- **Putnam’s work** (1976) as a key dynamic multivariable model for software cost and effort estimation. Putnam's model is based on Norden's work on manpower distribution, utilizing a Rayleigh curve to relate development effort and time to project size (often in KLOC).

Putnam's Work:

- In 1976, Putnam studied the problem of staffing of software projects:
  - observed that the level of effort required in software development efforts has a similar envelope.
  - **found that the Rayleigh-Norden curve**
    - ✓ **relates the number of delivered lines of code to effort and development time.**

Putnam's Work (CONT.)

- Putnam analyzed a large number of army projects, and derived the expression:

  $L = C_k$ K pow (1/3). td pow (4/3)

  - K is the effort expended (in PM) and L is the size in KLOC.
  - td is the time to develop the software.
  - C_k is the state of technology constant
    - ✓ reflects factors that affect programmer productivity.

Putnam's Work (CONT.)

- Ck=2, for poor development environment
  - ✓no methodology, poor documentation, and review, etc.
- Ck=8, for good software development environment
  - ✓software engineering principles used
- Ck=11, for an excellent environment

Rayleigh Curve

- Very small number of engineers are needed at the beginning of a project
  - carry out planning and specification.
- As the project progresses:
  - more detailed work is required,
  - number of engineers slowly increases and reaches a peak.

Rayleigh Curve

- Putnam observed that:
  - the time at which the Rayleigh curve reaches its maximum value
    - ✓corresponds to system testing and product release.
  - After system testing,
    - ✓the number of project staff falls till product installation and delivery.

Rayleigh Curve

- From the Rayleigh curve observe that:
  - approximately 40% of the area under the Rayleigh curve is to the left of td
  - and 60% to the right.

Effect of Schedule Change on Cost

- Using the Putnam's expression for L,
  K=L pow(3)/(Ck) pow(3) * td pow (4)
  Or, K=C1/(td) pow (4)
- For the same product size,

C1=L pow(3)/(Ck) pow (3) is a constant.

- Or, K1/K2 = (td_2) pow (4)/(td_1) pow (4)

✓ It is observed, When the schedule of a project is compressed, the required effort increases in proportion to the fourth power of the degree of compression

Effect of Schedule Change on Cost

(CONT.)

- Observe:
  - a relatively small compression in delivery schedule
    - ✓ can result in substantial penalty on human effort.
- Also, observe:
  - benefits can be gained by using fewer people over a somewhat longer time span.

Example

- If the estimated development time is 1 year, then in order to develop the product in 6 months,
  - the total effort and hence the cost increases 16 times.
  - In other words,
    - ✓the relationship between effort and the chronological delivery time is highly nonlinear.

Effect of Schedule Change on Cost (CONT.)

- Putnam model indicates extreme penalty for schedule compression
  - and extreme reward for expanding the schedule.
- Putnam estimation model works reasonably well for very large systems,
  - but seriously overestimates the effort for medium and small systems.

Effect of Schedule Change on Cost (CONT.)

- Boehm observed:
  - "There is a limit beyond which the schedule of a software project cannot be reduced by buying any more personnel or equipment."
  - This limit occurs roughly at 75% of the nominal time estimate for small and medium size project.

Effect of Schedule Change on Cost (CONT.)

- If a project manager accepts a customer demand to compress the development time by more than 25%
  - very unlikely to succeed.
    - every project has only a limited amount of parallel activities
    - sequential activities cannot be speeded up by hiring any number of additional engineers.
    - many engineers have to sit idle.

Jensen Model

- The Jensen model, is a software cost estimation model similar to the Putnam model, focusing on predicting development effort and time.
- It is designed to handle software development projects where schedule compression increases effort, notably assuming that effort increases proportionally to the square of the degree of compression.

Jensen Model

- Jensen model is very similar to Putnam model.
  - attempts to soften the effect of schedule compression on effort
  - to makes it applicable to smaller and medium sized projects.

Jensen Model

- Jensen proposed the equation:
  - $L = C_{te} t_d * K$ pow (1/2)
  - Where,
    - ✓ C_{te} is the effective technology constant,
    - ✓ t_d is the time to develop the software, and
    - ✓ K is the effort needed to develop the software.

Organization and Team Structure

- **Organization and team structures refer to how people and tasks are arranged to deliver software projects successfully.**
- **Organization Structure** involves dividing development efforts into **functional or project-based team's structure**.
- **Effective teams structure** require clear roles, planned communication, and appropriate sizing (typically small for better communication) to manage complexities like invisibility, changeability, and the need for high-quality, maintainable software.

Organization and Team Structure

- **Functional Organization:**
  - Engineers are organized into functional groups, e.g.
    - **specification, design, coding, testing, maintenance, etc.**
  - Engineers from functional groups get assigned to different projects

Advantages of Functional Organization

- High Specialization
- Easier Manage of staffing turnover
- Better documentation is produced
  - ✓different phases are carried out by different teams of engineers.
- Helps identify errors earlier.

Project based team Organization

- **A team is assigned to a single project, containing all necessary skills for that project.**
- Engineers get assigned to a project for the entire duration of the project
  - ✓Same set of engineers carry out all the phases
- **Advantages:**
  - ✓Engineers save time on learning details of every project.
  - ✓Leads to job rotation
- **Disadvantage:** **Generally, not suitable for small organizations with few projects due to inefficient resource usage**

Team Structure

- How individual teams are organized internally to manage communication and authority?
- Problems of different complexities and sizes require different team structures:
  - **Chief-programmer team**
  - **Democratic team**
  - **Mixed organization**

Democratic (Egoless) Team:

- No permanent leader; decisions are made by consensus, making it ideal for solving complex problems through shared ideas.

Democratic Teams

- **Suitable for:**
  - small projects requiring less than five or six engineers
  - research-oriented projects
- A manager provides administrative leadership:
  - at different times different members of the group provide technical leadership.

Democratic Teams

- **Democratic organization provides**
  - higher morale and job satisfaction to the engineers
  - therefore, leads to less employee turnover.
- **Suitable for less understood Problems**
  - a group of engineers can invent better solutions than a single individual.

Democratic Teams

- **Disadvantage:**
  - team members may waste a lot time arguing about trivial points:
    - absence of any authority in the team.

Democratic Teams contd..

- For large sized projects, a pure democratic organization tends to become chaotic.
- The democratic team organization encourages **egoless programming** as programmers can share and review one another's work.

Chief Programmer Team:

- A highly centralized structure where a senior "**chief programmer**" makes all technical decisions, supported by a backup programmer and specialized staff like librarians or testers.

Chief Programmer Team

- A senior engineer provides technical leadership:
  - partitions the task among the team members.
  - verifies and integrates the products developed by the members.

Chief Programmer Team

- Works well when
  - the task is well understood
    - also, within the intellectual grasp of a single individual,
  - importance of early completion outweighs other factors
    - team morale, personal development, etc.

Chief Programmer Team

- Chief programmer team is subject to Single point failure:
  - too much responsibility and authority is assigned to the chief programmer.

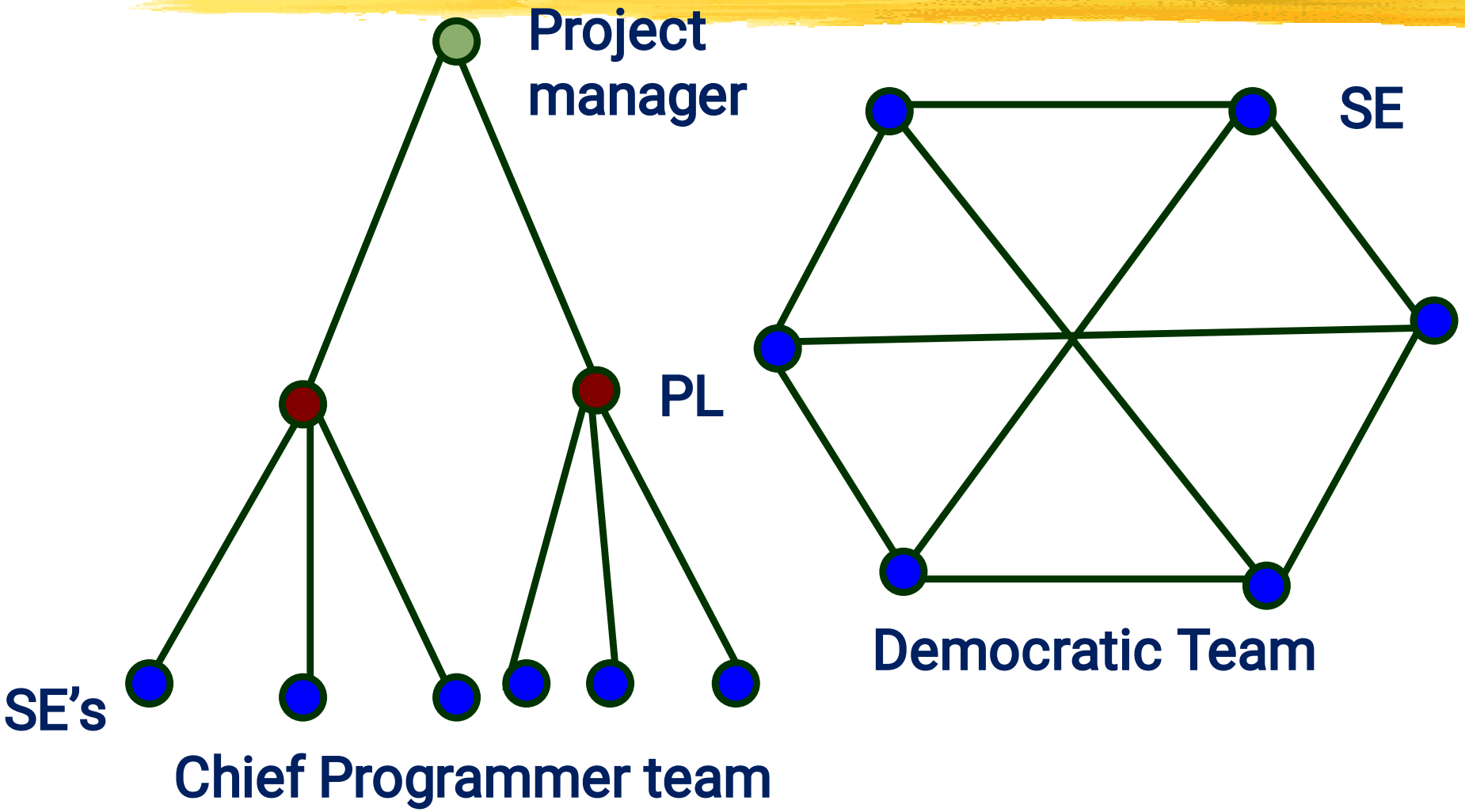
Mixed Control Model:

- **Combines both types**, using a democratic approach for senior-level problem-solving and a hierarchical structure for junior staff to ensure efficient task execution.

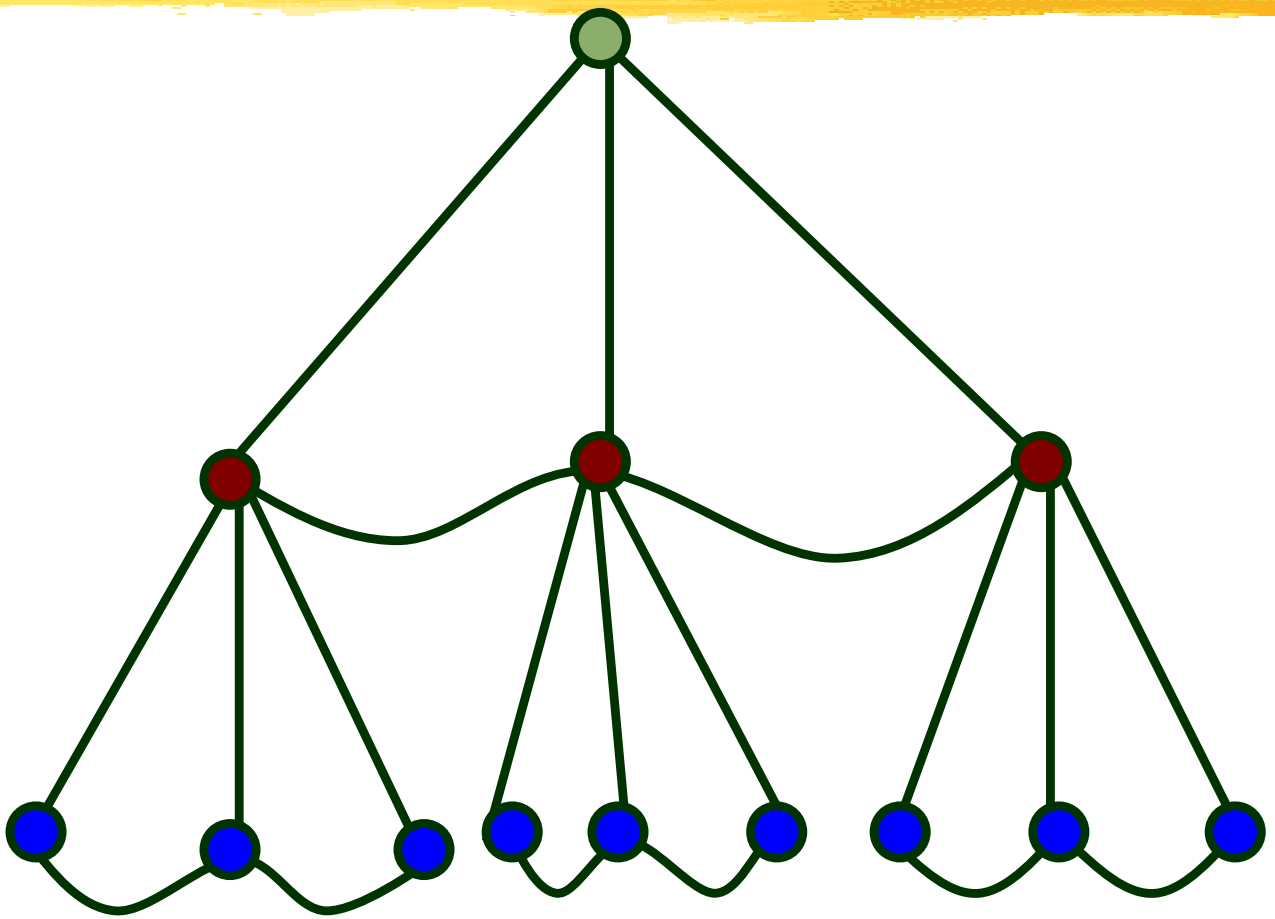
Mixed Control Team Organization

- **Draws upon ideas from both:**
  - democratic organization and
  - chief-programmer team organization.
- **Communication is limited**
  - to a small group that is most likely to benefit from it.
- **This team structure is extremely popular and is being used in many s/w dev. Companies and also Suitable for large organizations.**

Team Organization



Mixed team organization



Comparison

Comparison Summary

Feature	Democratic Team	Mixed Team
Control	Decentralized (Egoless)	Mixed (Decentralized sub-groups + Centralized lead)
Leadership	Rotational/Shared	Fixed coordination for sub-groups
Communication	High (All-to-all)	High within sub-groups; structured between them
Best Usage	Small, R&D, complex problems	Large, complex, multi-component projects

Summary

- We discussed the broad responsibilities of the project manager:
  - Project planning
  - Project Monitoring and Control

Summary

- To estimate software cost:
  - Determine size of the product.
  - Using size estimate,
    - ✓determine effort needed.
  - From the effort estimate,
    - ✓determine project duration, and cost.

Summary (CONT.)

- Cost estimation techniques:
  - Empirical Techniques
  - Heuristic Techniques
  - Analytical Techniques
- Empirical techniques:
  - based on systematic guesses by experts.
    - ✓Expert Judgement
    - ✓Delphi Estimation

Summary (CONT.)

- **Heuristic techniques:**
  - assume that characteristics of a software product can be modelled by a mathematical expression.
  - COCOMO
- **Analytical techniques:**
  - derive the estimates starting with some basic assumptions:
  - Halstead's Software Science

Summary (CONT.)

- The staffing level during the life cycle of a software product development:
  - follows Rayleigh curve
  - maximum number of engineers required during testing.

Summary (CONT.)

- **Relationship between schedule change and effort:**
  - highly nonlinear.
- **Software organizations are usually organized in:**
  - functional format
  - project format

Summary (CONT.)

- **Project teams can be organized in following ways:**
  - **Chief programmer:** suitable for routine work.
  - **Democratic:** Small teams doing R&D type work
  - **Mixed:** Large projects

Thank you!